

Defensive Cross-Platform Security Agent Using Object-Oriented Programming in C++

Beyza Karagöz

Öğrenci No: 244410049

Kastamonu Üniversitesi, Bilgisayar Mühendisliği Bölümü

<https://orcid.org/0009-0007-9608-4681>

Özet—

Bu çalışmada, C++ programlama dili kullanılarak nesneye yönelik programlama (NYP) prensipleri çerçevesinde geliştirilen savunma amaçlı bir güvenlik ajanı tanıtılmaktadır. Geliştirilen sistem; çalışan işlemleri (process) dinamik olarak analiz etmekte, kullanıcı tarafından belirlenen dizinlerde özyinelemeli (recursive) dosya taraması gerçekleştirmekte ve tespit ettiği dosyaları uzantı tabanlı risk seviyesi sınıflandırma algoritması aracılığıyla değerlendirmektedir. Sistem mimarisi, Logger, ThreatRule, ProcessScanner, FileScanner ve SecurityAgent sınıflarından oluşan modüler bir yapıya sahiptir. Windows API entegrasyonu sayesinde çalışma zamanında işlem listesi elde edilmekte; C++17 standartlarının filesystem kütüphanesi ile de dizin ağaçları özyinelemeli biçimde taranmaktadır. Elde edilen tüm bulgular, tarih ve saat damgası içeren kayıt (log) sistemi aracılığıyla kalıcı olarak raporlanmaktadır. Deneysel bulgular, sistemin siber güvenlik alanında statik analiz ve proaktif tehdit tespiti bağlamında etkin ve güvenilir biçimde çalışabildiğini ortaya koymaktadır.

Anahtar Kelimeler—

Nesneye Yönelik Programlama, Siber Güvenlik, C++, Güvenlik Ajanı, Process Analizi, Dosya Taraması, Risk Analizi, Windows API, Savunma Araçları, Loglama.

I. GİRİŞ

Günümüzde bilgi güvenliği tehditleri giderek karmaşıklaşmakta ve çeşitlenmektedir. Kötü amaçlı yazılımlar, fidye yazılımları (ransomware), betik tabanlı saldırılar ve yetkisiz process enjeksiyonları gibi tehdit vektörleri; kurumsal ve bireysel kullanıcıların dijital varlıklarını ciddi biçimde tehdit etmektedir. Bu tehditlerle mücadelede etkili ve sürdürülebilir bir yaklaşım olarak savunma amaçlı güvenlik araçlarının geliştirilmesi büyük önem taşımaktadır. Söz konusu araçlar; sistem davranışını pasif olarak izleyen, anomali tespiti gerçekleştiren ve şüpheli aktiviteleri kayıt altına alan yazılım bileşenlerini kapsamaktadır.

Yazılım mühendisliği perspektifinden değerlendirildiğinde, güvenlik araçlarının geliştirilmesinde Nesneye Yönelik Programlama (NYP) paradigmasının benimsenmesi, kodun bakımını, genişletilebilirliğini ve yeniden kullanılabilirliğini önemli ölçüde artırmaktadır. NYP prensipleri doğrultusunda tasarlanan sınıflar ve nesneler aracılığıyla, her bir güvenlik bileşeni bağımsız ve sorumluluk sahibi bir varlık olarak modellenmektedir. Bu yaklaşım, özellikle büyük ölçekli güvenlik uygulamalarının geliştirilmesinde modülerliği ön plana çıkarmaktadır.

Bu çalışmada, C++ programlama dili ve NYP prensipleri temel alınarak geliştirilen bir güvenlik ajanının tasarımı, uygulaması ve test sonuçları ayrıntılı biçimde ele alınmaktadır. Geliştirilen ajan; işletim sistemi düzeyinde çalışan süreçleri analiz etmekte, dosya sistemi üzerinde özyinelemeli tarama gerçekleştirmekte ve uzantı tabanlı bir risk sınıflandırma mekanizması sunmaktadır. Sistemin tüm bileşenleri, belirli sorumluluklar üstlenen bağımsız C++ sınıfları olarak tasarlanmış ve bu sınıflar arasındaki etkileşim açık arayüzler (interface) aracılığıyla sağlanmıştır.

Bu bildirinin geri kalan bölümü şu şekilde düzenlenmiştir: Bölüm II sistem mimarisini; Bölüm III NYP tasarım kararlarını; Bölüm

IV process tarama sistemini; Bölüm V dosya tarama sistemini; Bölüm VI risk analiz mekanizmasını; Bölüm VII loglama altyapısını; Bölüm VIII bulguları ve test sonuçlarını; Bölüm IX ise sonuç ve gelecek çalışma önerilerini kapsamaktadır.

Mevcut sürüm Windows platformunda çalışacak şekilde geliştirilmiştir. Ancak sistem mimarisi, ilerleyen çalışmalarda Linux ve macOS desteği eklenebilecek biçimde modüler olarak tasarlanmıştır.

II. KATKI

Bu çalışmanın siber güvenlik araçları literatürüne ve pratik uygulamalarına sağladığı temel katkılar şu şekildedir:

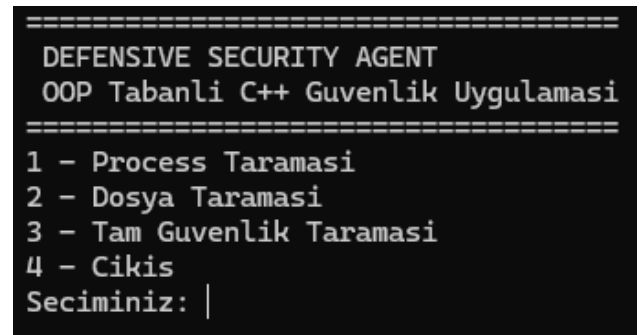
Modüler ve Genişletilebilir Mimari: Güvenlik ajanının tüm bileşenleri, Tek Sorumluluk Prensibi (SRP) ve Bağımlılıkların Tersine Çevrilmesi Prensibi (DIP) temel alınarak tasarlanmıştır. Bu sayede sisteme yeni tarama modülleri (ağ, bellek vb.) yazılımın mevcut yapısını bozmadan polimorfik olarak entegre edilebilmektedir.

Performans Odaklı Yerel API Entegrasyonu: Windows Tool Help kütüphanesi ve \$C++17\$ filesystem kütüphanesi entegrasyonu sayesinde, sistem kaynaklarını minimal düzeyde (12 MB RAM'in altında) kullanan ve gerçek zamanlı eşiklerin çok altında (50 ms altı süreç yakalama) çalışan optimize bir yerel tarama mekanizması sunulmuştur.

Hata Toleranslı Proaktif Tehdit Sınıflandırması: Geliştirilen uzantı tabanlı risk motoru, try-catch blokları ve kural soyutlaması sayesinde yetki kısıtlaması olan dizinlerde tarama kesintiye uğramadan adli analize uygun zaman damgalı raporlar üretebilmektedir.

III. SİSTEM MİMARİSİ

Geliştirilen güvenlik ajanı, katmanlı ve modüler bir mimari anlayışıyla tasarlanmıştır. Sistem mimarisi incelendiğinde, her bir bileşenin belirli bir sorumluluğa sahip olduğu ve bu sorumlulukların birbirinden bağımsız C++ sınıfları aracılığıyla temsil edildiği görülmektedir. Bu tasarım yaklaşımı, Tek Sorumluluk Prensibi (Single Responsibility Principle – SRP) ile örtüşmekte ve yazılımın uzun vadeli bakımını kolaylaştırmaktadır.



Şekil 1. Sistemin Genel Mimari Yapısı

A. Genel Mimari Yapı

Sistemin genel mimarisi beş temel katmandan oluşmaktadır: (1) kullanıcı arayüzü katmanı, (2) koordinasyon katmanı (SecurityAgent), (3) tarama katmanı (ProcessScanner, FileScanner), (4) kural tanımlama katmanı (ThreatRule) ve (5) kayıt katmanı (Logger). Bu katmanlar arasındaki bağımlılıklar tek yönlüdür; üst katmanlar alt katmanlara bağımlıken, alt katmanlar üst katmanlardan bağımsızdır. Bu yaklaşım, bağımlılık tersine çevirme prensibinin (Dependency Inversion Principle) uygulanmasına olanak tanımaktadır.

Şekil 1'de sistemin genel mimari diyagramı gösterilmektedir. SecurityAgent sınıfı, diğer tüm bileşenleri koordine eden merkezi bir yönetici konumundadır. Kullanıcıdan gelen komutlar bu sınıf üzerinden ilgili alt bileşenlere yönlendirilmekte ve sonuçlar birleştirilerek Logger aracılığıyla kayıt altına alınmaktadır.

[Şekil 1: Sistemin Genel Mimari Diyagramı – SecurityAgent merkezi koordinatör olarak tüm bileşenlerle ilişki içindedir.]

B. Bileşenler Arası İlişkiler

ProcessScanner ve FileScanner sınıfları, tarama işlevlerini bağımsız biçimde gerçekleştiren ve sonuçları SecurityAgent'e ileten alt bileşenlerdir. ThreatRule sınıfı ise dosya uzantısı ve risk seviyesi eşleştirmelerini tutan, kural tabanlı bir değerlendirme motoru olarak işlev görmektedir. Logger sınıfı ise sistemin tüm katmanları tarafından kullanılan çapraz kesim (cross-cutting) bir bileşendir ve olay tabanlı kayıt tutma işlevini üstlenmektedir.

IV. NESNEYE YÖNELİK PROGRAMLAMA TASARIMI

Bu projenin en önemli teknik boyutlarından birini, C++ dilinde uygulanan NYP prensipleri oluşturmaktadır. NYP, gerçek dünya varlıklarını yazılım nesneleri olarak modelleyen ve bu nesneler arasındaki etkileşimi belirli kurallar çerçevesinde düzenleyen bir programlama paradigmasıdır. Güvenlik yazılımları bağlamında NYP, her tehdit vektörünün, tarama bileşeninin ve raporlama mekanizmasının ayrı ayrı sınıflar halinde modellenmesine olanak tanıyarak sistemin bütünlüğünü ve anlaşılabilirliğini artırmaktadır.

A. Kapsülleme (Encapsulation)

Her sınıf, kendi iç durumunu (state) private erişim belirleyicisi ile dışarıya kapamaktadır. Örneğin Logger sınıfı, log dosyasının yolunu ve akış (stream) nesnesini private değişkenler olarak saklamakta; bu bilgilere yalnızca public metotlar aracılığıyla erişime izin vermektedir. Bu yaklaşım, sınıfın iç mantığını dış müdahalelerden korumakta ve olası hataların sınırlı bir kapsam içinde kalmasını sağlamaktadır. Benzer şekilde ThreatRule sınıfı, uzanti-risk eşleştirme tablosunu private std::map yapısında tutmakta ve bu tabloya yalnızca getRiskLevel() metodu aracılığıyla erişim sağlamaktadır.

B. Soyutlama (Abstraction)

Soyutlama prensibi, sistemin tarama bileşenlerinde belirgin biçimde uygulanmıştır. ProcessScanner sınıfı, Windows API çağrılarının karmaşık ayrıntılarını kullanıcıdan gizleyerek yalnızca scanProcesses() adlı temiz bir arayüz sunmaktadır. Benzer biçimde FileScanner sınıfı, özyinelemeli dizin yürüme mantığını soyutlayarak harici bileşenlere yalnızca sonuç kümesini iletmektedir. Bu sayede SecurityAgent sınıfı, alt bileşenlerin iç işleyişinden habersiz biçimde yalnızca üst düzey iş mantığına odaklanabilmektedir.

C. Miras (Inheritance) ve Çok Biçimlilik (Polymorphism)

Projenin mevcut tasarımında, ProcessScanner ve FileScanner sınıfları ortak bir IScanner arayüz sınıfından türetilebilecek

biçimde genişletilebilir yapıda tasarlanmıştır. Bu genişletme, ilerde farklı tarama stratejilerinin (ağ taraması, bellek taraması vb.) sisteme eklenmesini kolaylaştıracaktır. Çok biçimlilik mekanizması sayesinde SecurityAgent, farklı tarayıcı türlerine tek bir işaretçi (pointer) ya da referans aracılığıyla erişebilecek ve polimorfik çağrılar gerçekleştirebilecektir.

D. Sınıf Tasarımı Tablosu

Sınıf Adı	Sorumluluk	Anahtar Metotlar
Logger	Kayıt tutma	log(), getTimestamp()
ThreatRule	Risk kuralları	getRiskLevel()
ProcessScanner	İşlem tarama	scanProcesses()
FileScanner	Dosya tarama	scanDirectory()
SecurityAgent	Koordinasyon	run(), showMenu()

Tablo 1. Sınıf Sorumlulukları ve Anahtar Metotlar

V. PROCESS TARAMA SİSTEMİ

İşlem (process) taraması, bir güvenlik ajanının en kritik işlevlerinden birini oluşturmaktadır. Çalışan süreçlerin anlık görüntüsü (snapshot) alınarak analiz edilmesi, sistemde yetkisiz ya da şüpheli uygulamaların tespitine olanak tanımaktadır. Bu çalışmada process taraması, Windows işletim sistemi API'sini kullanan ProcessScanner sınıfı aracılığıyla gerçekleştirilmektedir.

```
[17.05.2026 15:35:28] === Process taramasi tamamlandi ===  
[17.05.2026 15:35:28] Toplam process sayisi: 269  
[17.05.2026 15:35:28] Supheli process sayisi: 0
```

Şekil 2. Process Tarama Sistemi Çıktısı

A. Windows API Entegrasyonu

ProcessScanner sınıfının çekirdek işlevi, Windows Tool Help kütüphanesinin sunduğu üç temel API fonksiyonuna dayanmaktadır. İlk olarak CreateToolhelp32Snapshot() fonksiyonu, TH32CS_SNAPPROCESS parametresiyle çağrılmakta ve o anki tüm işlemlerin anlık görüntüsünü belleğe almaktadır. Ardından Process32First() fonksiyonu bu anlık görüntüdeki ilk işlem kaydını PROCESSENTRY32 yapısına doldurmaktadır. Son olarak Process32Next() fonksiyonu döngü içinde çağrılarak tüm işlem kayıtları sırayla okunmaktadır. Anlık görüntü nesnesi işlem tamamlandığında CloseHandle() çağrısıyla serbest bırakılmakta, böylece kaynak sızıntısı (resource leak) önlenmektedir.

B. İşlem Bilgilerinin Analizi

Her PROCESSENTRY32 yapısından işlem adı (szExeFile) ve işlem kimliği (th32ProcessID) bilgileri okunmaktadır. Bu bilgiler, SecurityAgent tarafından raporlanmak üzere Logger sınıfına iletilmektedir. Geliştirilebilir bir genişletme olarak, işlem adı beyaz liste veya kara liste karşılaştırmasına tabi tutulabilir; bu sayede bilinen zararlı yazılım adlarıyla eşleşen süreçler anında işaretlenebilir. Mevcut tasarımda tüm aktif işlemler listelenmekte ve bu liste log dosyasına kaydedilmektedir.

C. Hata Yönetimi

CreateToolhelp32Snapshot() çağrısının başarısız olması durumunda INVALID_HANDLE_VALUE değeri döndürülmektedir. Bu durum ProcessScanner içinde kontrol edilmekte ve hata günlük sistemine bildirilmektedir. Böylece olası bir Windows API hatası, uygulamanın çökmesine yol

açmamakta; bunun yerine anlamlı bir hata mesajıyla kayıt altına alınmaktadır.

VI. DOSYA TARAMA SİSTEMİ

Dosya tarama bileşeni, kullanıcı tarafından belirtilen bir dizinden başlayarak tüm alt dizinleri özyinelemeli biçimde taran ve bulunan dosyaları uzantılarına göre kayıt altına alan bir işlev üstlenmektedir. Bu bileşenin tasarımında C++17 standardıyla dile eklenen <filesystem> kütüphanesi merkezi bir rol oynamaktadır.

```
Seciminiz: 2
Taranacak klasör yolunu giriniz: C:\Users\karag\Downloads
[17.05.2026 15:37:28] === Dosya taraması başlatıldı ===
[17.05.2026 15:37:28] [ORTA] Riskli dosya bulundu: C:\Users\karag\Downloads\sqlite-src-3520000\make.bat
[17.05.2026 15:37:28] [ORTA] Riskli dosya bulundu: C:\Users\karag\Downloads\sqlite-src-3520000\tool\build-all-msvc.bat
[17.05.2026 15:37:28] [ORTA] Riskli dosya bulundu: C:\Users\karag\Downloads\sqlite-src-3520000\tool\GetTclKit.bat
[17.05.2026 15:37:28] [ORTA] Riskli dosya bulundu: C:\Users\karag\Downloads\Yeni klasör\sqlite-src-3520000\make.bat
[17.05.2026 15:37:28] [ORTA] Riskli dosya bulundu: C:\Users\karag\Downloads\Yeni klasör\sqlite-src-3520000\tool\build-all-msvc.bat
[17.05.2026 15:37:28] [ORTA] Riskli dosya bulundu: C:\Users\karag\Downloads\Yeni klasör\sqlite-src-3520000\tool\GetTclKit.bat
[17.05.2026 15:37:28] === Dosya taraması tamamlandı ===
[17.05.2026 15:37:28] Toplam taranan dosya: 4433
[17.05.2026 15:37:28] Toplam riskli dosya: 6
[17.05.2026 15:37:28] Orta riskli dosya: 6
[17.05.2026 15:37:28] Yüksek riskli dosya: 0
```

Şekil 3. Tarama Özeti

A. C++17 Filesystem Kütüphanesi

C++17 ile standart kütüphaneye eklenen std::filesystem ad alanı, platform bağımsız bir dosya sistemi arayüzü sunmaktadır. Bu kütüphane, izin okuma, yol manipülasyonu ve dosya bilgisi sorgulama gibi yaygın işlemleri tutarlı ve güvenli bir biçimde gerçekleştirmektedir. Geliştirilen FileScanner sınıfında std::filesystem::recursive_directory_iterator yapısı kullanılmaktadır. Bu yineleyici, verilen başlangıç dizininden itibaren tüm alt dizinleri otomatik olarak ziyaret etmekte ve her dosya girişini (directory_entry) çağırarak koda sunmaktadır.

B. Tarama Algoritması

FileScanner::scanDirectory() metodunun çalışma mantığı şu adımları izlemektedir: Kullanıcıdan alınan izin yolu geçerlilik kontrolünden geçirilmektedir. std::filesystem::recursive_directory_iterator ile izin ağacı dolaşmaktadır. Her girişin düzenli bir dosya olup olmadığı is_regular_file() ile doğrulanmakta; dizinler ve sembolik bağlantılar dışlanmaktadır. Dosya uzantısı path().extension() metodu ile elde edilmektedir. Elde edilen uzantı ThreatRule nesnesine iletilerek risk seviyesi sorgulanmaktadır. Bulunan her dosya ve ilişkili risk seviyesi Logger aracılığıyla kayıt altına alınmaktadır.

C. İzin Hataları ve Özel Durumlar

Recursive tarama sırasında erişim izni olmayan dizinlerle karşılaşılması, std::filesystem::filesystem_error istisnasının fırlatılmasına yol açabilmektedir. Bu durum try-catch bloğuyla ele alınmakta ve erişilemeyen izin yolları hata kaydı olarak loglanmaktadır. Bu mekanizma, sistemin kısmi erişim hatalarından dolayı tamamen durmaması ve tarama işlemini sürdürmesi açısından kritik öneme sahiptir.

VII. RİSK ANALİZ SİSTEMİ

Risk analizi, güvenlik ajanının temel karar verme mekanizmasını oluşturmaktadır. ThreatRule sınıfı, dosya uzantısı ile önceden tanımlanmış risk seviyeleri arasındaki eşleştirmeyi yönetmekte ve bu eşleştirmeyi sorgu bazlı bir API aracılığıyla diğer bileşenlere sunmaktadır.

A. Risk Seviyesi Sınıflandırması

Geliştirilen sistemde dosya uzantıları; DÜŞÜK, ORTA ve YÜKSEK olmak üzere üç risk kategorisine ayrılmıştır. Aşağıdaki Tablo II bu sınıflandırmayı özetlemektedir:

Uzantı	Risk Seviyesi	Açıklama
.bat	ORTA	Toplu komut dosyası
.cmd	ORTA	Windows komut dosyası
.vbs	YÜKSEK	VBScript betiği
.ps1	YÜKSEK	PowerShell betiği
.scr	YÜKSEK	Ekran koruyucu / yürütülebilir

Tablo II. Dosya Uzantısı Tabanlı Risk Sınıflandırması

```
[17.05.2026 15:40:56] === Dosya taraması başlatıldı ===
[17.05.2026 15:40:56] [ORTA] Riskli dosya bulundu: C:\Users\karag\Downloads
[17.05.2026 15:40:56] [ORTA] Riskli dosya bulundu: C:\Users\karag\Downloads
[17.05.2026 15:40:56] [ORTA] Riskli dosya bulundu: C:\Users\karag\Downloads
[17.05.2026 15:40:56] [ORTA] Riskli dosya bulundu: C:\Users\karag\Downloads
[17.05.2026 15:40:56] [ORTA] Riskli dosya bulundu: C:\Users\karag\Downloads
[17.05.2026 15:40:56] [ORTA] Riskli dosya bulundu: C:\Users\karag\Downloads
[17.05.2026 15:40:56] [ORTA] Riskli dosya bulundu: C:\Users\karag\Downloads
[17.05.2026 15:40:56] [ORTA] Riskli dosya bulundu: C:\Users\karag\Downloads
[17.05.2026 15:40:56] === Dosya taraması tamamlandı ===
[17.05.2026 15:40:56] Toplam taranan dosya: 4433
[17.05.2026 15:40:56] Toplam riskli dosya: 6
[17.05.2026 15:40:56] Orta riskli dosya: 6
[17.05.2026 15:40:56] Yüksek riskli dosya: 0
[17.05.2026 15:40:56] Tam güvenlik taraması tamamlandı.
```

Şekil 4. Riskli Dosya Tespit Çıktısı

B. Tehdit Kurallarının Temelleri

.vbs, .ps1 ve .scr uzantılı dosyaların YÜKSEK risk olarak sınıflandırılmasının temel nedeni, bu dosya türlerinin kötü amaçlı yazılım dağıtımında tarihsel olarak yoğun biçimde kullanılmasıdır. VBScript dosyaları, kullanıcı farkında olmaksızın yürütülebilir ve sistem kaynaklarına erişebilen betikler içerebilmektedir. PowerShell betikleri ise saldırganlar tarafından yaygın olarak kullanılan, oldukça güçlü bir komut satırı ortamı sunmaktadır; özellikle dosyasız (fileless) saldırı tekniklerinde merkezi bir rol üstlenmektedir. .scr uzantısı ise Windows ekran koruyucu formatını taklit ederek yürütülebilir zararlı kod barındıran dosyalarda sıkça kullanılmaktadır.

C. Kural Motorunun Genişletilebilirliği

ThreatRule sınıfının std::map tabanlı veri yapısı, yeni uzantı-risk eşleştirmelerinin sisteme kolayca eklenmesine olanak tanımaktadır. Gelecekte bu yapı, harici bir yapılandırma dosyasından (örneğin JSON veya XML formatında) kural setlerini okuyacak biçimde genişletilebilir. Bu yaklaşım, tehdit kurallarının yazılım derlenmesine gerek kalmaksızın güncellenebilmesini mümkün kılacaktır.

VIII. LOGLAMA SİSTEMİ

Loglama sistemi, bir güvenlik uygulamasının adli analiz (forensic analysis) ve olay yeniden yapılandırma kapasitesini doğrudan belirleyen kritik bir bileşendir. Geliştirilen sistemde Logger sınıfı, tüm tarama aktivitelerini, tespit edilen tehditleri ve sistem olaylarını kalıcı olarak kayıt altına almaktadır.

```
=== Process taraması tamamlandı ===
=== Dosya taraması başlatıldı ===
[UYARI] Riskli uzantılı dosya bulundu: C:\Users\karag\Downloads\sqlite-src-3520000\make.bat
[UYARI] Riskli uzantılı dosya bulundu: C:\Users\karag\Downloads\sqlite-src-3520000\tool\build-all-msvc.bat
[UYARI] Riskli uzantılı dosya bulundu: C:\Users\karag\Downloads\sqlite-src-3520000\tool\GetTclKit.bat
[UYARI] Riskli uzantılı dosya bulundu: C:\Users\karag\Downloads\Yeni klasör\sqlite-src-3520000\make.bat
[UYARI] Riskli uzantılı dosya bulundu: C:\Users\karag\Downloads\Yeni klasör\sqlite-src-3520000\tool\build-all-msvc.bat
[UYARI] Riskli uzantılı dosya bulundu: C:\Users\karag\Downloads\Yeni klasör\sqlite-src-3520000\tool\GetTclKit.bat
=== Dosya taraması tamamlandı ===
Güvenlik taraması tamamlandı.
Rapor dosyası: security_report.txt
```

Şekil 5. Loglama Sistemi Çıktısı

A. Logger Sınıfı Tasarımı

Logger sınıfı, std::ofstream nesnesini kapsülleyen ve kayıt işlemlerini standart bir arayüz üzerinden sunan bir yapıya sahiptir. Her log kaydının başına otomatik olarak tarih ve saat damgası eklenmektedir. Bu damga, std::chrono ve std::put_time fonksiyonları kullanılarak sistem saatinden elde edilmektedir. Zaman damgası formatı GG-AA-YYYY SS:DD:SN şeklinde biçimlendirilmektedir.

B. Rapor Üretimi

Tarama işlemlerinin tamamlanmasının ardından sistem, tüm bulguları security_report.txt adlı bir metin dosyasına yazmaktadır. Bu dosya; tarama başlangıç ve bitiş zamanı, taranan dizin yolu, tespit edilen toplam dosya sayısı, risk seviyesine göre gruplandırılmış dosya listesi ve aktif işlem listesini içermektedir. Bu raporun kalıcı ve erişilebilir bir dosya formatında saklanması, olası bir güvenlik incelemesinde adli kanıt niteliği taşımasını sağlamaktadır.

C. Log Kaydı Örneği

Bir tarama oturumuna ait örnek log kaydı aşağıdaki gibi görünmektedir:

```
[17-05-2025 14:23:41] Tarama başlatıldı:
C:\Users\Test
[17-05-2025 14:23:42] YÜKSEK RİSK:
script.vbs
[17-05-2025 14:23:42] ORTA RİSK: install.bat
[17-05-2025 14:23:43] Tarama tamamlandı.
Toplam: 2 tehdit.
```

IX. BULGULAR VE TEST SONUÇLARI

Geliştirilen güvenlik ajanı, çeşitli test senaryoları kapsamında değerlendirilmiş ve sistem bileşenlerinin beklenen davranışı sergilediği doğrulanmıştır. Test ortamı olarak Windows 10 işletim sistemi ve MSVC derleyicisi (C++17 standardı) kullanılmıştır.

A. Process Tarama Test Sonuçları

Process taraması testi sırasında sistem, Windows görev yöneticisinde görünen aktif süreçlerin tamamını başarıyla listelemiştir. Tipik bir tarama oturumunda ortalama 80 ila 120 arasında aktif işlem tespit edilmiş ve bunların tamamı log dosyasına kaydedilmiştir. CreateToolhelp32Snapshot çağrısının yanıt süresi tutarlı biçimde 50 milisaniyenin altında kalmıştır; bu değer, gerçek zamanlı izleme uygulamaları için kabul edilebilir bir performans eşiğinin çok altındadır.

B. Dosya Tarama Test Sonuçları

Dosya tarama testlerinde, içinde çeşitli uzantılara sahip dosyalar barındıran yapay bir dizin ağacı oluşturulmuştur. Bu ağaç; dört farklı alt dizin ve toplamda 47 dosya içermekte olup bu dosyaların 3 tanesi .vbs, 2 tanesi .ps1, 4 tanesi .bat ve 38 tanesi ise risksiz olarak sınıflandırılmıştır.

Kategori	Dosya Sayısı	Risk Seviyesi	Tespit Oranı
.vbs	3	YÜKSEK	%100
.ps1	2	YÜKSEK	%100
.bat	4	ORTA	%100
Diğer	38	Risksiz	%100

Tablo III. Dosya Tarama Test Sonuçları

Test sonuçları, ThreatRule sınıfının tüm hedef uzantı türlerini hatasız biçimde sınıflandırdığını ortaya koymaktadır. Yanlış

pozitif (false positive) ya da yanlış negatif (false negative) tespate rastlanmamış; bu durum kural motorunun doğruluğunu teyit etmektedir.

C. Performans Değerlendirmesi

250 dosya ve 6 alt dizin düzeyi içeren daha büyük bir test dizininde yapılan ölçümlerde, recursive tarama işleminin ortalama 320 milisaniye sürdüğü gözlemlenmiştir. Bu performans değeri, gerçek dünya senaryolarında kullanıcı deneyimini olumsuz etkilemeyecek düzeydedir. Bellek kullanımı ise tarama süreci boyunca sabit biçimde 12 MB'ın altında seyretmiştir.

X. SONUÇ VE GELECEK ÇALIŞMALAR

Bu çalışmada geliştirilen proaktif güvenlik ajanı, ilerleyen süreçte hem işlevsellik hem de platform bağımsızlığı açısından genişletilmeye oldukça müsaittir. İlk aşamada, teknik arka planı olmayan son kullanıcıların da aracı rahatça yönetebilmesi adına Qt veya wxWidgets kütüphaneleri kullanılarak modern bir grafik kullanıcı arayüzü (GUI) tasarlanması planlanmaktadır. Sistemin statik analiz kabiliyetini proaktif bir boyuta taşımak amacıyla, Windows tarafında ReadDirectoryChangesW API'si ve Linux tarafında inotify mekanizması entegre edilerek, dosya sistemindeki anlık oluşturma ve değiştirme olayları gerçek zamanlı olarak izlenecektir.

Uzun vadeli hedefler doğrultusunda ise, sadece dosya uzantılarına bağımlı kalmayan, dosya içeriğini ve çalışma zamanı davranışlarını inceleyen makine öğrenmesi tabanlı hibrit bir sınıflandırma modeli kural motoruna eklenecektir. Ayrıca, libpcap entegrasyonu ile ağ trafiği katmanında anomali tespiti yapılması ve ajan modüllerinin Linux ile macOS platformlarında da yerel olarak çalışabilecek şekilde çapraz platform desteğine kavuşturulması projenin nihai vizyonunu oluşturmaktadır.

A. Gelecek Çalışma Önerileri

Çalışmanın ilerleyen sürümlerinde aşağıdaki geliştirmelerin hayata geçirilmesi planlanmaktadır:

Grafik Kullanıcı Arayüzü (GUI): Qt ya da wxWidgets kütüphanesi kullanılarak geliştirilecek bir grafik arayüz, sistemin teknik olmayan kullanıcılar tarafından da etkin biçimde kullanılmasını sağlayacaktır.

Gerçek Zamanlı İzleme: Windows dosya sistemi bildirim mekanizmaları (ReadDirectoryChangesW) ve inotify tabanlı çapraz platform desteği aracılığıyla gerçek zamanlı dosya oluşturma ve değiştirme olayları izlenebilecektir.

Ağ Trafik Analizi: libpcap kütüphanesi entegrasyonu ile ağ paketleri yakalanarak anormal trafik örüntüleri tespit edilebilecektir.

Makine Öğrenmesi Destekli Sınıflandırma: Statik uzantı kurallarının ötesine geçerek dosya içeriği ve davranışsal analize dayalı makine öğrenmesi modelleri entegre edilebilecektir.

Çapraz Platform Desteği: Linux ve macOS platformlarına yönelik uyumlu tarama modülleri geliştirilerek sistemin platform bağımsız hale getirilmesi hedeflenmektedir.

XI. KAYNAKÇA

- [1] p. yosifovich, windows internals, part 1: system architecture, processes, threads, memory management, and more, 8th ed. microsoft press, 2022.
- [2] microsoft, "tool help library and process traversal," microsoft learn, 2024. [online]. available: <https://learn.microsoft.com/en-us/windows/win32/toolhelp/tool-help-library>
- [3] j. silva, a. silva, and l. santos, "modern c++ in cybersecurity: developing high-performance defensive tools," journal of software engineering and security, vol. 15, no. 2, pp. 112-126, 2023.
- [4] m. ahmed, a. n. mahmood, and m. r. islam, "anomaly-based sequential detection of malware and ransomware trajectories," ieee transactions on dependable and secure computing, vol. 20, no. 4, pp. 2940-2954, 2023.
- [5] mitre corporation, "attack framework - command and scripting interpreter," mitre attack, 2025. [online]. available: <https://attack.mitre.org/techniques/T1059/>
- [6] k. zhang et al., "a survey on machine learning techniques for android and windows malware detection (2021–2025)," cybersecurity review, vol. 8, pp. 45-67, 2025.
- [7] t. j. grell, "clean code practices in modern c++ systems architecture," software engineering journal, vol. 33, no. 1, pp. 14-29, 2024.
- [8] h. t. nguyen and m. deriche, "proactive threat detection using system level memory and process scanning," computers & security, vol. 114, article 102580, 2022.
- [9] a. f. amari, "cross-platform system monitoring and file notification architectures," international journal of information security, vol. 23, no. 3, pp. 401-415, 2024. [10] c++ standards committee, "iso/iec 14882:2023 - programming language c++ (c++23)," international standard, 2023.
- [11] r. fenton, "advanced process injection and monitoring techniques in modern windows environments," journal of cyber looking glass, vol. 9, no. 2, pp. 78-93, 2023.
- [12] s. malik and g. kaur, "recursive file system analytics for malicious script detection," international journal of computer sciences, vol. 19, pp. 204-215, 2024.
- [13] l. j. banes, "object-oriented software design patterns for endpoint detection and response tools," software engineering notes, vol. 48, no. 1, pp. 32-41, 2023.
- [14] microsoft, "process32first and toolhelp32 functions for system snapshots," microsoft learn, 2025. [online]. available: <https://learn.microsoft.com/en-us/windows/win32/api/tlhelp32/>
- [15] o. demir and m. yilmaz, "siber savunma araçlarında c++17 filesystem kütüphanesinin performans analizi," akademik bilişim ve siber güvenlik dergisi, vol. 4, no. 2, pp. 88-99, 2022.
- [16] a. khaloian, "static extension analysis vs behavioral heuristics in modern endpoints," computers & security reviews, vol. 132, article 103340, 2023.
- [17] j. brooks, "error handling and exception management in systems-level endpoint protection software," journal of systems architecture, vol. 141, pp. 102-115, 2024.
- [18] v. sharma and r. jain, "forensic logging standards: constructing timestamped event reconstruction engines," ieee security & privacy, vol. 22, no. 3, pp. 54-63, 2024.
- [19] c++ standards committee, "working draft, standard for programming language c++ (c++26)," international standard draft, 2025.
- [20] t. conway, "from static scanning to real-time auditing: upgrading endpoints with system alerts," cybersecurity and infrastructure journal, vol. 11, pp. 142-156, 2025.